Event ends in 1 day 4 hours 5 minutes.

Why Custom Agents?

You now have rules (steering) and procedures (skills). But right now, Kiro's default agent handles everything. What if you want a dedicated agent identity specifically for EOL reporting, with its own system prompt and configuration? That's what custom agents are for. In Section 3, you'll see advanced agent patterns like `includePowers` and parallel subagent invocation.

Custom agents provide a way to customize Kiro behavior by defining specific configurations for different use cases. Each custom agent is defined by a configuration file that specifies which tools the agent can access, what permissions it has, and what context it should include.

Learn more: CLI Custom Agents docs

You now have rules (steering) and procedures (skills). But right now, Kiro's default agent handles everything. What if you want a dedicated agent identity specifically for EOL reporting, with its own system prompt and configuration? That's what custom agents are for.

Custom agents allow you to:

- **Pre-approve specific tools** — Define which tools can run without prompting
- **Limit tool access** — Restrict which tools are available to reduce complexity
- **Include relevant context** — Automatically load project files, documentation, or system information
- **Configure tool behavior** — Set specific parameters for how tools should operate

When to Use Each Layer

Scenario	Use
General rules for all conversations	Steering
Reusable procedure for a specific task	Skill
Dedicated identity with specific behavior	Agent
Different agents for different domains	Agent per domain

Step 3.1: Create the Agent File

Custom agents in Kiro are markdown files stored in `.kiro/agents/` (workspace) or `~/.kiro/agents/` (global). Each agent has:

- A **description** in YAML frontmatter — tells Kiro when to auto-select this agent
- A **system prompt** in the markdown body — defines the agent's identity, behavior, and instructions

Create the agents directory:

```
mkdir -p .kiro/agents
```

IDE (.md) | CLI (.json)

Create `.kiro/agents/eol-reporting.md`:

```
cat > .kiro/agents/eol-reporting.md << 'EOF'
---
name: eol-reporting
description: >
  AI agent specialized in tracking End of Service (EOS) and End of Life (EOL) dates
  for AWS cloud infrastructure resources. Use when asked to generate EOL reports,
  check deprecation dates, inventory AWS resource versions, or assess lifecycle risks.
tools: ["read", "write", "shell", "web"]
model: auto
includeMcpJson: >
  {
    "aws-knowledge-mcp": {
      "url": "https://knowledge-mcp.global.api.aws",
      "disabled": false,
      "autoApprove": ["aws___search_documentation", "aws___read_documentation"]
    }
  }
---

# EOS/EOL Reporting Agent

You are an AI agent specialized in tracking EOS/EOL dates for cloud infrastructure resources.

## Core Workflow
1. Inventory AWS Resources — Use AWS CLI
2. Retrieve EOL Data — Use the `retrieve-eol-data` skill
3. Categorize by Urgency — EXPIRED / Critical / Warning / Monitor
4. Generate Reports — Use branded templates

## Rules
- AWS official documentation is always the primary source
- Format dates consistently (YYYY-MM-DD)
- Include resource identifiers (ARN, name, region)

## Data Source Priority
1. AWS Official Documentation (via Knowledge MCP)
2. AWS Health Dashboard
3. AWS CLI APIs
4. endoflife.date API (non-AWS only)

## Report Templates
- Markdown: #[[file:.kiro/agents/references/eol-report/eol-report-template.md]]
- HTML: #[[file:.kiro/agents/references/eol-report/eol-report-template.html]]
EOF
```

Key Points

- The `description` in YAML frontmatter is what Kiro uses for auto-selection — include trigger phrases like "EOL report", "deprecation dates"
- The markdown body is the system prompt — it defines the agent's identity and full instructions
- `includeMcpJson` embeds the MCP server config directly in the agent file

Step 3.2: Refactor the Steering File

Now that the agent owns the full rulebook, the steering file should slim down to just a routing pointer:

```
cat > .kiro/steering/eol-reporting.md << 'EOF'
---
inclusion: always
---
# EOS/EOL Reporting Agent

For any End of Service (EOS) and End of Life (EOL) topics — including generating EOL reports,
checking deprecation dates, inventorying AWS resource versions, or assessing lifecycle risks —
you must use the `eol-reporting` custom agent.
EOF
```

Why keep the steering file? It's always loaded into every conversation. By keeping a slim pointer, any conversation will know to route EOL questions to the right agent.

Step 3.3: Move Report Templates

Since the agent now owns the report configuration, move templates to the agent's references folder:

```
mkdir -p .kiro/agents/references/eol-report
mv .kiro/steering/eol-report/* .kiro/agents/references/eol-report/
rmdir .kiro/steering/eol-report
```

Step 3.4: Test the Agent

Before testing, start a fresh session so Kiro picks up the new agent.

IDE (.md) | CLI (.json)

Kiro should auto-select the `eol-reporting` agent when your request matches its description:

```
Generate a weekly EOL report for my AWS infrastructure
```

You can also invoke it explicitly using the agent selector in the chat panel.

Step 3.5: Verify the Complete Architecture

Your final directory structure should look like this:

```
.kiro/
├── agents/
│   ├── eol-reporting.md          # Agent identity & prompt (IDE)
│   ├── eol-reporting.json        # Agent config for CLI
│   └── references/
│       └── eol-report/
│           ├── eol-report-template.html
│           ├── eol-report-template.md
│           └── logo-2c2p-d.webp
├── skills/
│   └── retrieve-eol-data/
│       ├── SKILL.md
│       └── references/
│           └── data-sources.md
└── steering/
    └── eol-reporting.md          # Agent routing pointer
```

Test the complete flow:

```
Generate a weekly EOS/EOL report for my AWS infrastructure. Include:
1. Summary section with counts by category
2. Critical resources section (< 6 months)
3. Warning resources section (6-12 months)
4. Monitor resources section (1-2 years)
5. Recommendations section with prioritized actions
```

The agent owns the rules and report format. The steering routes EOL questions to the agent. The skill provides the procedures. All three layers work together seamlessly.

Final Architecture

```
┌─────────────────────────────────────┐
│ Agent (.kiro/agents/eol-reporting.md)  │
│ "Who does the work + what are the rules" │
│ + Identity, rules, report format, templates │
│ + Auto-selected or invoked explicitly  │
├─────────────────────────────────────┤
│ Steering (.kiro/steering/eol-reporting.md) │
│ "Route to the right agent"          │
│ + Always loaded into every conversation │
│ + Points to eol-reporting custom agent │
├─────────────────────────────────────┤
│ Skill (.kiro/skills/retrieve-eol-data/) │
│ "How to do the task"                │
│ + Loaded on-demand when task matches │
│ + Step-by-step procedures, API calls │
│ + references/ loaded only as needed │
└─────────────────────────────────────┘
```

Final Directory Structure

```
.kiro/
├── agents/
│   ├── eol-reporting.md          # Agent identity (IDE)
│   ├── eol-reporting.json        # Agent config (CLI)
│   └── references/
│       └── eol-report/
│           ├── eol-report-template.html
│           ├── eol-report-template.md
│           └── logo-2c2p-d.webp
├── skills/
│   └── retrieve-eol-data/        # Reusable skill
│       ├── SKILL.md
│       └── references/
│           └── data-sources.md
└── steering/
    └── eol-reporting.md          # Agent routing pointer
```

The Story Arc

1. **Steering** — "I taught Kiro the rules" → Everything works, but the file is bloated with both rules and procedures
2. **Skills** — "I packaged the procedures so they load only when needed" → Context is lean, procedures are reusable
3. **Custom Agent** — "I built a dedicated agent that owns the rules and ties it all together" → A purpose-built identity with full rulebook, steering becomes a routing pointer

Troubleshooting

Issue	Solution
Agent can't access AWS Knowledge MCP	Verify `~/.kiro/settings/mcp.json` configuration
Skill doesn't activate automatically	Check `description` field includes trigger phrases; start a fresh session
Agent isn't auto-selected	Check `description` in YAML frontmatter includes relevant keywords
EOL dates are incorrect	Verify AWS official documentation is being used as primary source
AWS Health API returns SubscriptionRequiredException	Account lacks Business/Enterprise Support — agent should continue with other sources

What's Next

You've built the complete 3-layer stack: Steering -> Skill -> Agent. In Section 3, you'll add a 4th layer (Powers) and learn advanced patterns for each layer using a Terraform infrastructure review use case.